

A Project Report On

WEB SCRAPER & DATA EXTRACTOR

Submitted in partial fulfillment of the requirement for the
award of the degree

Bachelor of Science (Information Technology)
(B.Sc IT)

Academic Year 2026 – 27

Vishakha Mundhava (92400588029)
Manasvi Solanki (92400588040)
Trushita Shukla (92400588045)

Internal Guide
Prof. Tirth Bhadeshiya



Marwadi
University
Marwadi Chandarana Group



Rajkot-Morbi Road, At & PO : Gauridad, Rajkot 360 003. Gujarat. India.



**Bachelor of Science (Information Technology)
(B.Sc IT)**

Certificate

**This is to certify that the project work entitled
WEB SCRAPER & DATA EXTRACTOR
submitted in partial fulfillment of the requirement for
the award of the degree of
Bachelor of Science (Information Technology)
of the**

Marwadi University

is a result of the bonafide work carried out by

Vishakha Mundhava (92400588029)

Manasvi Solanki (92400588040)

Trushita Shukla (92400588045)

during the academic year 2026-27

Faculty Guide

HOD

Dean

DECLARATION

I/We hereby declare that this project work entitled **Web Scraper & Data Extractor** is a record done by me.

I also declare that the matter embodied in this project is genuine work done by me and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirement of any course of study.

Place :

Date :

Vishakha Mundhava (92400588029) Signature :

Manasvi Solanki (92400588040) Signature :

Trushita Shukla (92400588045) Signature :

CONTENTS

Chapters	Particulars	Page No.
1	Introduction to Project Definition	
2	PREAMBLE	
2.1	Module description	
3	REVIEW OF LITERATURE Analyze, Study and Compare the similar types of the systems/applications and highlights major findings	
4	TECHNICAL DESCRIPTION	
4.1	Hardware Requirement	
4.2	Software Requirement	
5	SYSTEM DESIGN AND DEVELOPMENT (Only applicable diagrams)	
5.1	• Algorithm	
5.2	• Flow Chart	
5.3	• Data Flow Diagram	
5.4	• Class Diagram	
5.5	• Use Case Diagram	
5.6	• Sequential Diagram	
5.7	• Activity Diagram	
5.8	• State Diagram	
5.9	• Database Design / File Structure (If applicable)	
5.10	• Relationship Diagram (ER)	
5.11	• Menu Design	
5.12	• Screen Design	
5.13	• Code of the module	
5.14		
6	SYSTEM TESTING	
7	CONCLUSION	
8	LEARNING DURING PROJECT WORK	
8.1	Future Enhancement	
9	BIBLIOGRAPHY	
9.1	Online References	
9.2	Offline References	

FIGURE INDEX

Sr. No.	Figure No.	Particulars	Page No.
1	Figure 1.1	Flow Chart	11
2	Figure 1.2	Data Flow Diagram	12
3	Figure 1.3	Class Diagram	13
4	Figure 1.4	Use Case Diagram	14
5	Figure 1.5	Sequential Diagram	15
6	Figure 1.6	Home Design	16
7	Figure 1.7	Scraped Data Screen	16
8	Figure 1.8	Search/filter option	16
9	Figure 1.9	CSV Download Screen	17

(Figure index must contain any types of Figures, Diagrams, Drawings, Screen Shots, etc.)

TABLE INDEX

Table No.	Particulars	Page No.
Table 1.1	Hardware Requirements	9
Table 1.2	Software Requirements	9
Table 1.3	File structure	11

(Table index must contain any types of Tabular information like Hardware / Software Requirement, Database / File Structure / Table Structure, Test Cases, etc.)

1. Introduction to Project Definition

Websites contain large amounts of information in HTML format. Manually collecting specific information from webpages can be repetitive and time-consuming. Web scraping provides a way to automatically retrieve and organize selected information from a webpage.

This project provides a simple educational web scraper. A user enters an accessible webpage URL. The system sends a request to the webpage, parses the returned HTML, extracts selected elements, and displays the results in an organized interface

2. PREAMBLE

The system uses Python libraries such as Requests, BeautifulSoup, Pandas, and Streamlit to fetch webpage content, parse HTML, extract useful information, display the results, and export the collected data in CSV format.

This project has helped us understand the practical concepts of web scraping, HTML parsing, data processing, Python modules, and interactive application development.

2.1 Module description

1. URL Input Module

- This module allows the user to enter the URL of the webpage that needs to be scrapped.
- The entered URL is passed to the scraping function for processing.

2. Webpage Fetching Module

- This module uses the **Requests** library to send an HTTP GET request and fetch the HTML content of the webpage.

3. HTML Parsing Module

- This module uses **BeautifulSoup** to parse the HTML content and make it easier to find and extract required webpage elements

4. Data Extraction Module

- This module extracts:

- Page title
- H1, H2 and H3 headings
- Links
- Image URLs

5. Search and Filter Module

- This module allows users to search the extracted information using a keyword and displays matching results.

6.Data Display Module

- This module uses **Streamlit** to display the extracted headings, links and images in separate tabs and tables.

7. CSV Export Module

- This module uses **Pandas** to organize the extracted data and provide it as a downloadable CSV file.

3. REVIEW OF LITERATURE

Web scraping is a technique used to automatically collect information from websites. Python provides libraries such as Requests for fetching webpages, BeautifulSoup for parsing HTML and extracting data, and Pandas for organizing data. Streamlit can be used to create an interactive interface for displaying the collected information. Based on these technologies, our project provides a simple way to extract and organize webpage data such as titles, headings, links, and images.

4. TECHNICAL DESCRIPTION

4.1 Hardware Requirements

Sr. No.	Hardware Component	Specification
---------	--------------------	---------------

1	Processor	Intel Core i3 or higher
2	RAM	4 GB or higher
3	Storage	10 GB free space
4	Internet	Required

Table 1.1

4.2 Software Requirements

Sr. No.	Software	Description
1	Operating System	Windows 10/11
2	Programming Language	Python 3.x
3	IDE	Visual Studio Code
4	Framework	Streamlit
5	Libraries	Requests, BeautifulSoup, Pandas
6	Web Browser	Google Chrome / Microsoft Edge
7	Output Format	CSV

Table 1.2

5. SYSTEM DESIGN AND DEVELOPMENT

5.1 Algorithm

- Start
- Accept the website URL from the user.
- Send an HTTP request to the given URL using Requests.
- Fetch the HTML content of the webpage.
- Parse the HTML using BeautifulSoup.
- Extract the page title, H1/H2/H3 headings, links, and image URLs.
- Convert relative URLs into complete URLs and remove duplicates.
- Display the extracted data using Streamlit.
- Allow the user to search/filter the extracted data.
- Prepare the data in CSV format using Pandas.
- Provide the CSV file for download.
- Stop.

5.2 Flow Chart

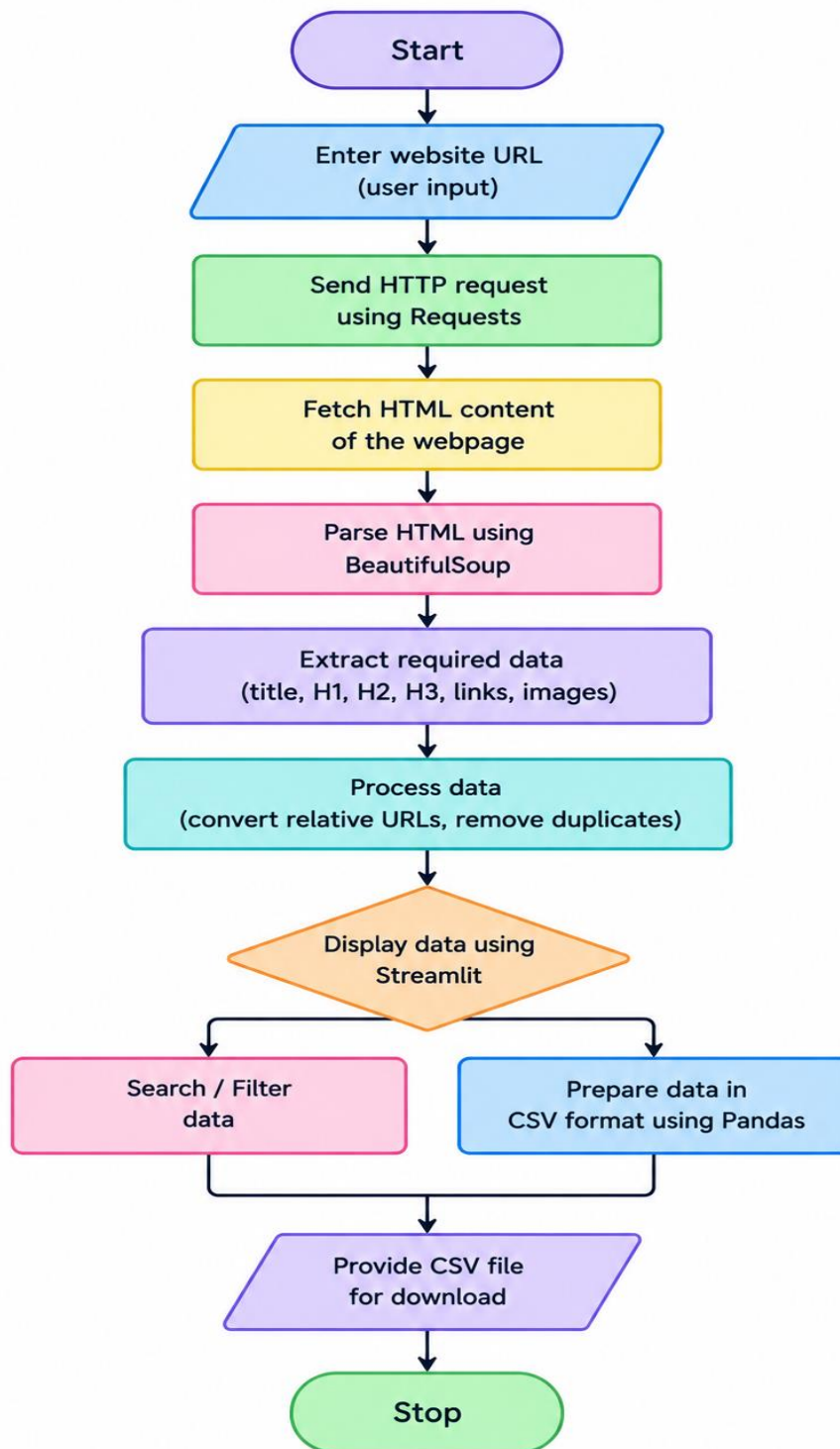


Figure 1.1

5.3 Data Flow Diagram (Level 0)

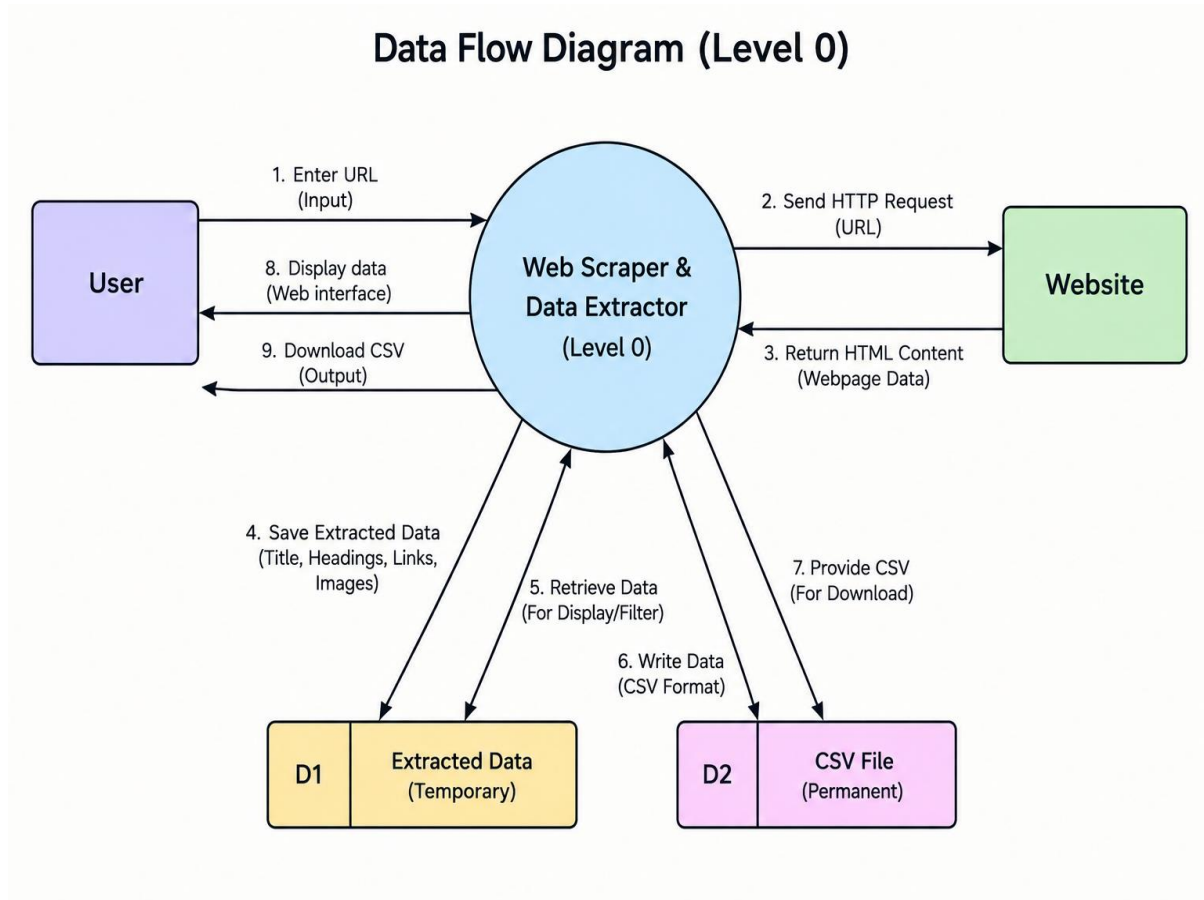


Figure 1.2

5.4 Class Diagram

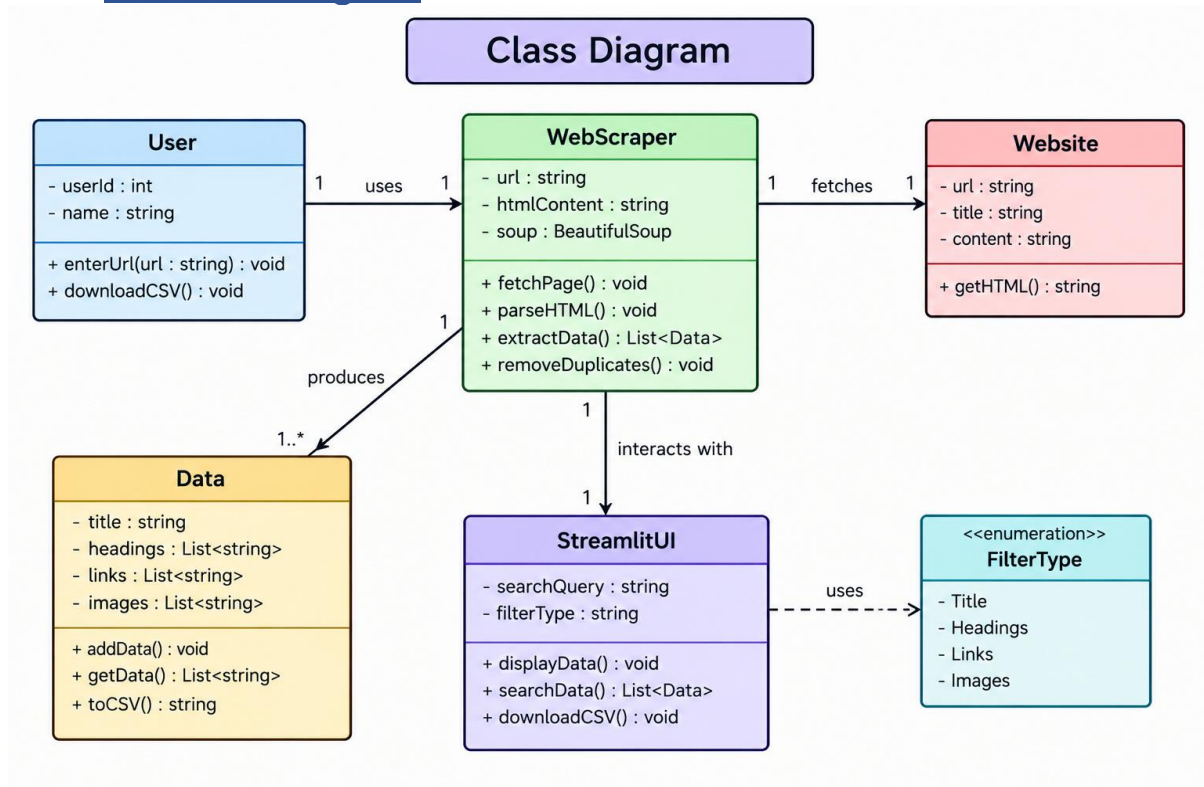


Figure 1.3

5.5 Use Case Diagram

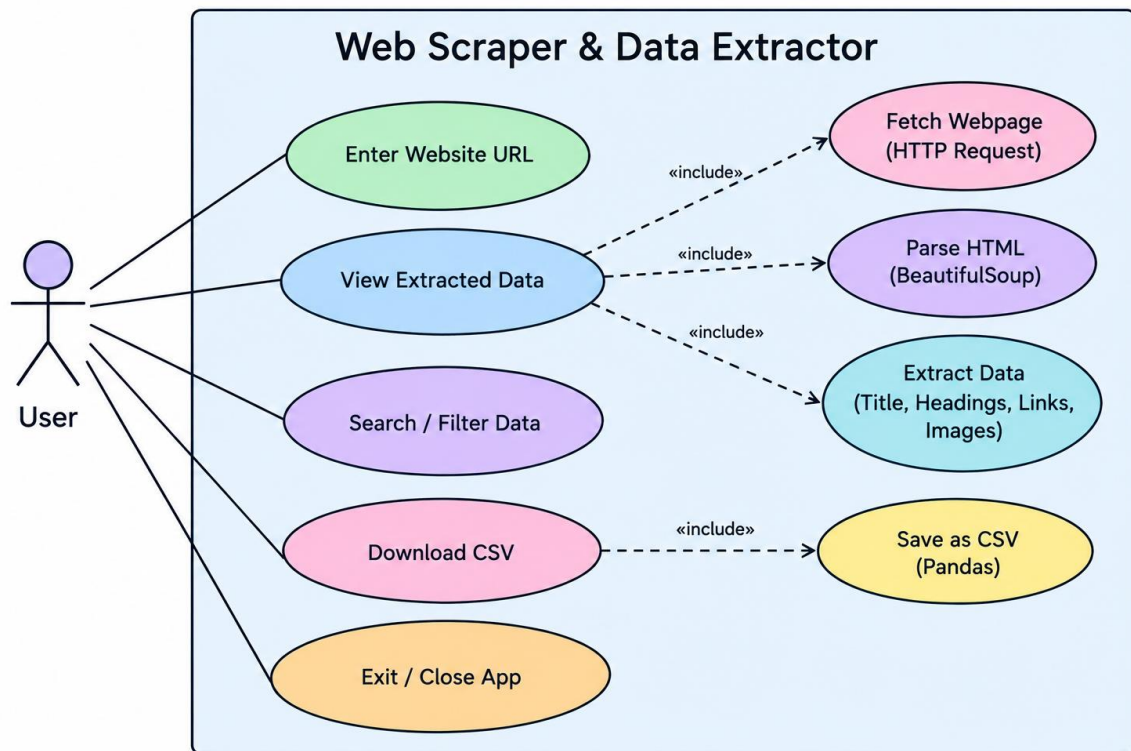


Figure 1.4

5.6 Sequential Diagram

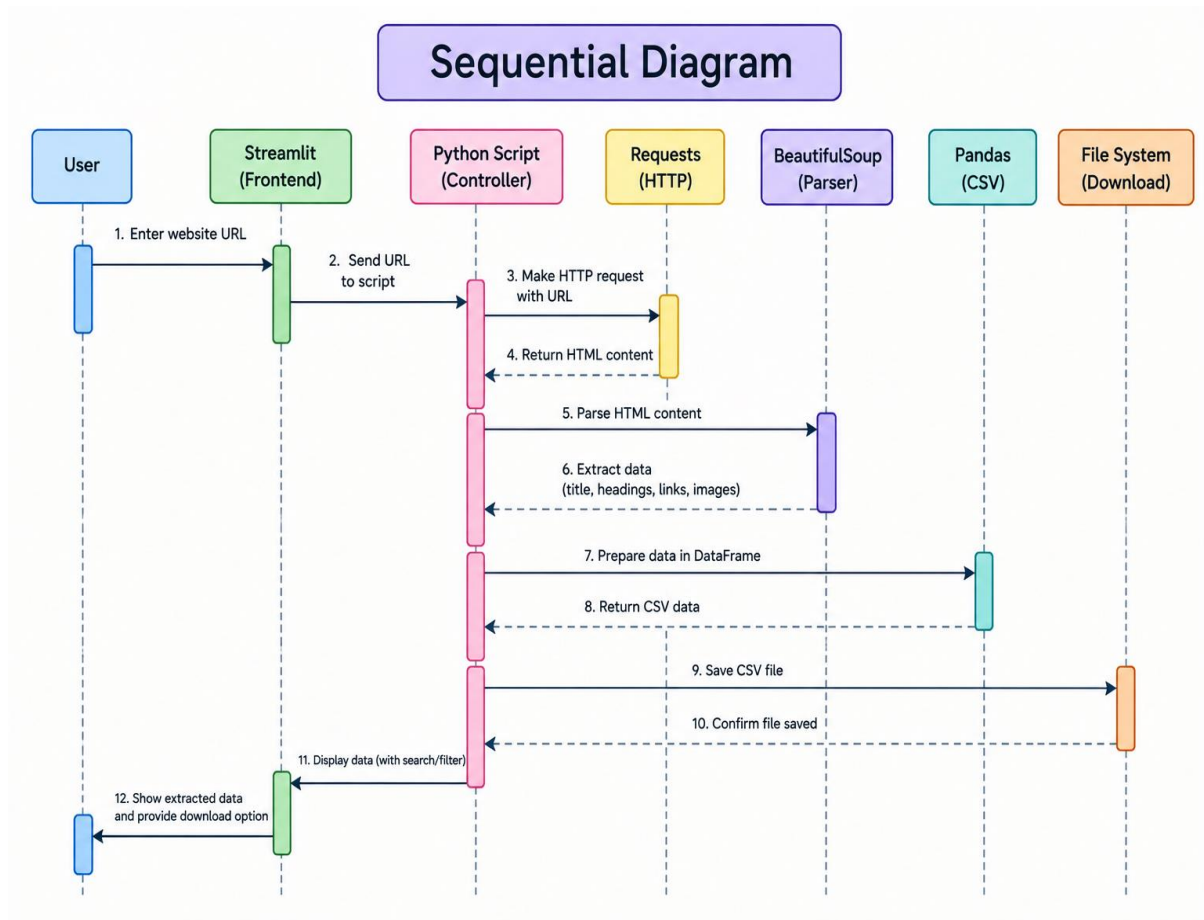


Figure 1.5

5.7 File Structure

File Name	Description
app.py	Contains the Streamlit user interface and main application logic.
scraper.py	Contains the web scraping and data extraction functions
requirements.txt	Contains the Python libraries required to run the project.
scraped_data.csv	Stores the extracted website data in CSV format.

Table 1.3

5.8 Screen Design

- Home screen

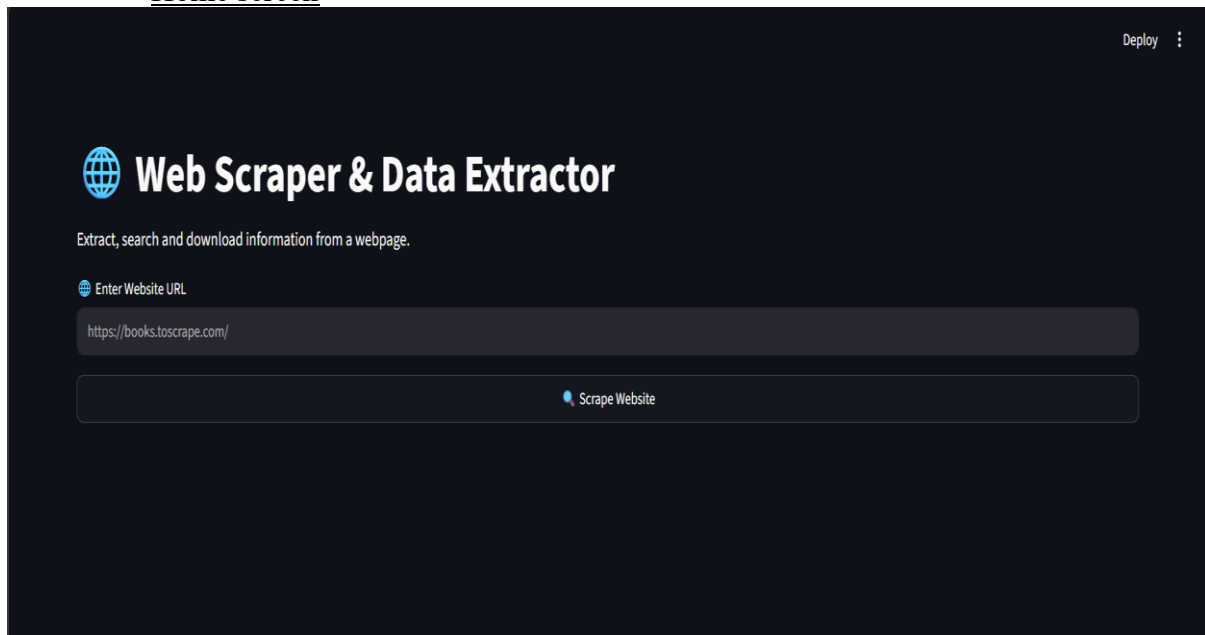


Figure 1.6

- Scraped Data Screen

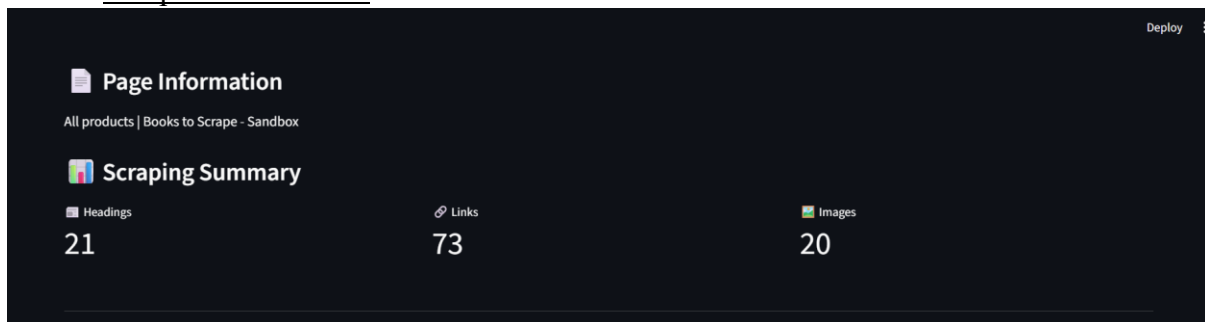


Figure 1.7

- Search/filter option



Figure 1.8

- CSV Download Screen



Download All Data as CSV

Figure 1.9

5.9 Code of the module

- [code files](#)

6. SYSTEM TESTING

The Web Scraper & Data Extractor was tested with different website URLs to ensure that the system correctly fetches webpages, extracts data, displays results, and generates CSV files.

7. CONCLUSION

The Web Scraper & Data Extractor successfully automates the process of collecting useful information from webpages. The system extracts page titles, headings, links, and images using Python, displays the results through Streamlit, and allows the data to be exported in CSV format. This project helped us understand practical concepts of web scraping, HTML parsing, data processing, and Python application development.

8. LEARNING DURING PROJECT WORK

During this project, we learned how to use Python for web scraping and data extraction. We gained practical knowledge of Requests, BeautifulSoup, Pandas, and Streamlit. We also learned how to parse HTML, extract useful webpage data, process it, and store the results in CSV format. Additionally, we improved our understanding of project development, testing, GitHub, and teamwork.

8.1 Future Enhancement

In the future, the project can be enhanced by adding support for JavaScript-based websites, automatic scheduled scraping, more advanced data filtering, and additional export formats such as Excel and JSON. The system can also be improved to handle larger amounts of data and provide more detailed data analysis.

9. BIBLIOGRAPHY

This project is developed with the help of various learning resources including books, online tutorials, and documentation.

9.1 Online References

The project development was supported by online references from the official Python website, Requests documentation, BeautifulSoup documentation, Pandas documentation, and Streamlit documentation. W3Schools and GeeksforGeeks were also referred to for learning Python, HTML, and web scraping concepts.

9.2 Offline References

- College lecture notes and study materials
- Reference books on Python programming
- Guidance from faculty and lab instructors